Graduate Algorithms

Quiz 5: Solutions

14 October, 2026

Question 1

(1+4) × 3 = 15 marks

Suggested/Proposed by Vardhan Kumar Ray

For each of the following subparts, you have to define the recurrence to fill the table T indexed as described in the subpart. You must also define the base cases that need to be filled before using the recurrence.

(a) Packing for the Inspector

A shady seller has a line of n products $x_1, x_2, \ldots, x_n$, each labelled `G` if it is genuine or `F` if it is fake, so the line can be written as a word over $\{\texttt{G}, \texttt{F}\}$. The seller must pack the entire line into boxes. Each box receives a *consecutive* stretch $x_i x_{i+1} \ldots x_j$ of the line, every product goes into exactly one box, and the order of the products is kept. A box holds at most c products, where $1 \le c \le n$. To avoid scrutiny, the seller makes sure that at least half of the products in every box are genuine, that is, no box contains more fakes than genuine products. We want the number of different ways to pack the whole line.

For example, take the line `FGGFG` with $c = 3$, writing a packing with a bar between consecutive boxes. There are exactly 4 valid packings:

$$\texttt{FG}\,|\,\texttt{GFG}, \qquad \texttt{FGG}\,|\,\texttt{FG}, \qquad \texttt{FG}\,|\,\texttt{G}\,|\,\texttt{FG}, \qquad \texttt{FG}\,|\,\texttt{GF}\,|\,\texttt{G}.$$

The packing `FGGF` | `G` is not valid, since its first box holds $4 > c$ products.

Suppose you are already provided with a 1D table $F[i]$ defined for all $0 \le i \le n$, where $F[i]$ is the number of fake products among $x_1, \ldots, x_i$ (so $F[0] = 0$). Define a 1D table $T[i]$ indexed by $i \in \{0, 1, \ldots, n\}$, where $T[i]$ stores the number of valid packings of the first i products $x_1, \ldots, x_i$, so that the required answer is $T[n]$. Write:

(i) The **base case(s)** for initializing $T[i]$.

(ii) The **recurrence relation** to compute $T[i]$ using smaller indices and F.

Solution.

Base case: $T[0] = 1$.

Recurrence: for $1 \le i \le n$,

$$T[i] \;=\; \sum_{\substack{\max(0,\, i-c) \le j \le i-1 \\ 2\,(F[i]-F[j]) \le i-j}} T[j],$$

where an empty sum is 0.

Check on the example: For `FGGFG` with $c = 3$,

i	0	1	2	3	4	5
x_i	–	F	G	G	F	G
$F[i]$	0	1	1	1	2	2
$T[i]$	1	0	1	2	1	4

so $T[5] = 4$, as stated.

(b) Distinct Probes

A DNA strand is a word $w = w_1 w_2 \dots w_m$ over the alphabet $\{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{T}\}$. A *probe* of length k binds to w if it can be obtained from w by deleting $m - k$ of its letters, not necessarily consecutive, without reordering the rest; that is, if it is a subsequence of w of length k. Two probes are the same if they are equal as words, no matter which positions of w they come from. For example, $w = \texttt{ACAC}$ has exactly 4 distinct probes of length 2, namely $\texttt{AA}$, $\texttt{AC}$, $\texttt{CA}$ and $\texttt{CC}$; the probe $\texttt{AC}$ arises from three different choices of positions but is counted once. Given w and an integer N with $0 \le N \le m$, we want the number of distinct probes of length N.

Suppose you are already provided with a 1D table $L[i]$ defined for all $1 \le i \le m$, where $L[i]$ is the largest index $j < i$ with $w_j = w_i$, and $L[i] = 0$ if no such j exists. Define a 2D table $T[i][k]$ indexed by $i \in \{0, 1, \dots, m\}$ and $k \in \{0, 1, \dots, N\}$, where $T[i][k]$ stores the number of distinct subsequences of length k of the prefix $w_1 w_2 \dots w_i$. The empty word counts as the unique subsequence of length 0 of every prefix, including the empty prefix. Write:

(i) The **base case(s)** for initializing $T[i][k]$.

(ii) The **recurrence relation** to compute $T[i][k]$ using smaller indices and L.

Solution.

Base cases: $T[i][0] = 1$ for $0 \le i \le m$, and $T[0][k] = 0$ for $1 \le k \le N$.

Recurrence: for $1 \le i \le m$ and $1 \le k \le N$,

$$T[i][k] \;=\; \begin{cases} T[i-1][k] + T[i-1][k-1] - T\Big[L[i]-1\Big][k-1], & \text{if } L[i] \ge 1, \\ T[i-1][k] + T[i-1][k-1], & \text{if } L[i] = 0. \end{cases}$$

Check on the example: For $w = \texttt{ACAC}$ we have $L = [0, 0, 1, 2]$, and

i	0	1	2	3	4
w_i	–	A	C	A	C
$T[i][0]$	1	1	1	1	1
$T[i][1]$	0	1	2	2	2
$T[i][2]$	0	0	1	3	4

so $T[4][2] = 4$, as stated.

(c) Bijection Balance

Let $\Sigma_1 = \{a_1, \ldots, a_k\}$ and $\Sigma_2 = \{b_1, \ldots, b_k\}$ be two disjoint alphabets, with a bijection $\psi : \Sigma_1 \to \Sigma_2$ given by $\psi(a_j) = b_j$. The alphabets and the bijection are fixed in advance; in particular, $k \geq 1$ is a constant independent of the input length.

Given a list of $m \geq 1$ non-empty strings $S_1, \ldots, S_m$ over $\Sigma_1 \cup \Sigma_2$, we want to decide whether it is possible to select exactly one letter from each string so that the set A of selected letters satisfies

$$\text{for every } j \in \{1, \ldots, k\}, \qquad \{a_j, b_j\} \not\subseteq A.$$

The same letter may be selected from more than one string. Thus, the only forbidden choice is selecting both letters of a matched pair, possibly from different strings.

Define a Boolean table $T[i][v]$, where $i \in \{0, 1, \ldots, m\}$ and $v \in \{0, 1, 2\}^k$. The entry $T[i][v]$ is true if and only if there is a valid selection from the first i strings whose selected letters are described *exactly* by v, where

$$v_j = \begin{cases} 0 & \text{if neither } a_j \text{ nor } b_j \text{ has been selected,} \\ 1 & \text{if } a_j \text{ has been selected,} \\ 2 & \text{if } b_j \text{ has been selected.} \end{cases}$$

For $t \in \{0, 1, 2\}$, write $v[j \leftarrow t]$ for the vector obtained by replacing coordinate j of v by t. Write $\ell \in S_i$ to mean that the letter ℓ occurs in the string S_i.

The required answer is true if and only if $T[m][v]$ is true for at least one state $v \in \{0, 1, 2\}^k$. Write:

(i) The **base case(s)** for initializing $T[i][v]$.

(ii) The **recurrence relation** to compute $T[i][v]$ using entries in row $i - 1$.

Solution.

Base cases: $T[0][(0, \ldots, 0)] = \text{true}$, and $T[0][v] = \text{false}$ for every $v \neq (0, \ldots, 0)$.

Recurrence: for $1 \leq i \leq m$ and $v \in \{0, 1, 2\}^k$,

$$T[i][v] \;=\; \bigvee_{\substack{j\,:\,v_j=1 \\ a_j \in S_i}} \Big(T[i-1][v] \vee T[i-1][v[j \leftarrow 0]]\Big) \;\vee\; \bigvee_{\substack{j\,:\,v_j=2 \\ b_j \in S_i}} \Big(T[i-1][v] \vee T[i-1][v[j \leftarrow 0]]\Big),$$

where an empty disjunction is false. In particular $T[i][(0, \ldots, 0)] = \text{false}$ for $i \geq 1$.

Check on a small example: Take $k = 1$, $S_1 = a_1$ and $S_2 = b_1 a_1$. Then

i	$T[i][(0)]$	$T[i][(1)]$	$T[i][(2)]$
0	true	false	false
1	false	true	false
2	false	true	false

so the answer is true: select a_1 from both strings.